

COS341 Compiler Construction ■ Semester Test

Tuesday the 8th of October 2024, Hall EMS-4-152, 14:30-16:30h

This question-booklet must be filled in and returned together with the answer-booklet



INSTRUCTIONS:

Read each task carefully before writing your answers *neatly* into the answer-booklet provided. Please **avoid** (as far as possible) to fill the answer-booklet with irrelevant rough-scribbles: instead, use your own rough-scribble pad for such purposes.

Wait until the invigilators give the *start-signal* before you begin to work.

Write with **indelible ink**.

You may solve the given tasks in any sequence that you prefer:

Begin with those tasks that you find most easy, and proceed from there to the more difficult tasks.

FORBIDDEN:

- Any kind of *academic dishonesty* (as defined by the rule book of the University of Pretoria);
- Any *electronic or computational devices* such as portable computers, "smart" cell-phones, "smart" wrist-watches, e-book display devices (e.g.: "Kindle"), and the like;
- The *textbook as a whole*;
- *Prints* from 3rd-party materials which you have not authored, including the E-book version of our textbook, photos of the lecture slides, internet wikipedia web-pages, scientific papers from GoogleScholar, etc.;
- Writing with *pencil that can be erased* by a rubber-gum.

ALLOWED:

- Student's own self-made (*hand-written*) crib-notes booklet, firmly bound with a solid spine (no "flying" pages), such as shown (for example) in the COS341 Study-Guide on page 4.

As per COS341 Study-Guide (page 7), this Semester Test has an **Assessment Value** of **20 Points**:

Task A.1:		[1 Point]
Task A.2:		[1 Point]
Task A.3:		[4 Points]
Task A.4:		[2 Points]
Task A.5:		[3 Points]
Task A.6:		[3 Points]
Task A.7:		[3 Points]
Task B :		[3 Points]

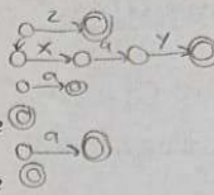
Timing Advice:

Since there are 8 Tasks for 2 hours of work-time, the *average work-time per task* is *approximately a quarter of an hour* (15 minutes). However, *some of the tasks can be solved much faster*, such that you will have ample time available for the "bigger" tasks.

Task A.

Given is the following small context-free grammar, G , (the production rules of which are numbered from 1 to 6 for easier reference):

1. $S \rightarrow Z$
2. $Z \rightarrow X a Y$
3. $X \rightarrow a$
4. $X \rightarrow \text{// nullable}$
5. $Y \rightarrow a$
6. $Y \rightarrow \text{// nullable}$



A.1) [1P]

Provide the given grammar's generated language $L(G)$ entirely.

A.2) [1P]

Prove by means of two structurally different syntax trees that G is ambiguous.

A.3) [4P]

For each of G 's Nonterminal symbols,

- *provide* its FIRST set,
- *provide* its FOLLOW set – Remember the additional rule number 0 needed for this purpose!

Present your results *neatly* in a *tabular* format:

Nonterminal symbol	FIRST set	FOLLOW set
⋮	⋮	⋮

A.4) [2P]

For all those rule-pairs of G , which have the same Nonterminal symbol on the left-hand-side, apply the method of the Look-Ahead-Sets to prove that G is not in the class LL(1): Identify and point out the relevant overlaps.

Present your results *neatly* in a *tabular* format:

Rule number	Look-Ahead-Set	Comments about <u>relevant</u> overlaps
⋮	⋮	⋮

SCENARIO:

Though our friend *Curt Curious* knows very well that **SLR parsing** is not possible if the underlying grammar is ambiguous, he would nevertheless try to *find out what would happen* if somebody would attempt to construct an SLR parse table for an ambiguous grammar – such as our given grammar G . In the following sub-tasks, you must help *Curt Curious* with the construction of the SLR parse-table for this grammar G – including the additional rule number **0** which is also needed for this purpose.

A.5) [3P]

Draw the non-deterministic control-NFA that corresponds to the structure of the given grammar G .

- **Advice:** In order to label the states of the NFA without causing confusion, use some Capital Letters that do not also appear as Nonterminal symbols of the given grammar (such as, for example: A, B, C, D, etc...)

A.6) [3P]
 Draw the **minimal deterministic finite automaton** DFA_{min} that is **language-equivalent** with the NFA from the foregoing sub-task.

- **Advice:** To **avoid confusion** with the numbered rules (1, 2, 3, etc...) of the given grammar G , please label the states of your DFA_{min} with **Roman numbers** (I, II, III, IV, V, VI, VII etc...)

A.7) [3P]
 On the basis of your DFA_{min} of above, **provide the given grammar's SLR Parsing-Table** including the parser actions *shift*, *reduce*, *go*, *accept*, and **highlight** specifically those table fields in which a **conflict** (due to the ambiguity of the given grammar) has appeared.

Advice:

- For those actions, which refer to DFA_{min} states, use the **Roman numbers** (I, II, III, etc...)
- For those actions, which refer to **grammar rules**, use the Arabic numbers (0, 1, 2, 3, etc...)

Task B. [3P]

There is a subtle *difference* in the *semantics* of a **do-while**-loop [**not** in our Textbook] in comparison against the *semantics* of a **repeat-until**-loop [which **is** in our Textbook in Figure 6.5]: Though both kinds of loops must "run" their body-program at least once, the **do-while**-loop **goes into its iteration (repetition)** if the boolean condition **still** holds at the end of the body-program, whereas the **repeat-until**-loop **exits the iteration as soon as** the boolean condition is true at the end of the body-program.

For **both** kinds of loops, our Textbook's translator function [from Textbook Figure 6.5] would create and emit a string of *Intermediate Code* that has the following structure:

LABEL L1 ++ bodyCode ++ boolCode ++ LABEL L2

whereby the **boolCode** string is a **result** from a **call** to the **Trans_cond** function in Textbook Fig.6.6.

In order to obtain the correct *Intermediate Code* for the **repeat-until**-loop, our Textbook [Figure 6.5] states that

boolCode = Trans_cond(COND, L2, L1, vtable, ftable)

How –with which parameters– must the same **Trans_cond** function [from Textbook Figure 6.6] be **called** if we wish to obtain correct *Intermediate Code* for a **do-while**-loop, instead of the Textbook's **repeat-until**-loop?

- **Write** your answer in the **form**: **boolCode = Trans_cond(... , ... , ... , ... , ...)** whereby you **replace the dots (...)** by the correct call-parameters.

- End of Semester Test Paper: there are no further questions -

